

# Java script

## JAVASCRIPT C'EST QUOI

- **JavaScript** est un langage de programmation coté client.
- C'est un langage qui va nous permettre de créer de l'**interaction** entre l'utilisateur et le client.
- Javascript est un langage de scripts qui s'incorpore au langage Html de présentation des pages Web.
- Les scripts vont permettre de rendre une page Web plus dynamique :
  - en animant certaines zones de la page,
  - ou en faisant réagir certains éléments de la page en fonction de certains événements provoqués par l'utilisateur.

## Rôle de JavaScript dans le web

- HTML: Structure de la page
- CSS: Mise en forme
- JavaScript: Interaction & logique

### **Exemple :**

- HTML : bouton
- CSS : couleur du bouton
- JS : action quand on clique

## Où placer le JavaScript ?

Le JavaScript peut être placé à trois endroits différents :

- Dans l'élément **head** d'une page HTML.
- Dans l'élément **body** d'une page HTML.
- Dans un fichier portant l'extension **.js** séparé.

## Dans l'élément head

- Dans le cas de l'insertion du code JavaScript à l'intérieur du head, il faut placer le JavaScript à l'intérieur d'une balise **script**.
- On place généralement l'élément script à la fin de notre élément head.

**<head>**

[...]

**<script>** alert("Au secours !!"); **</script>**

**</head>**

## Dans l'élément body

- Dans le cas de l'insertion du code JavaScript à l'intérieur du body, il faut aussi placer le JavaScript à l'intérieur d'une balise script.

**<script>** alert("Au secours !!"); **</script>**

## Dans un fichier JavaScript séparé

- Un fichier JavaScript a pour extension .js.
- est la méthode recommandée car elle permet une meilleure maintenabilité
- lier nos fichiers HTML et JavaScript en utilisant à nouveau un élément script et son attribut src.
- Si l'on souhaite lier un fichier JavaScript **script.js** à un fichier html placé dans le même dossier nous écrirons à la fin du head :
- `<script src="script.js"></script>` Dans le fichier **script.js**, on pourra écrire par exemple cette ligne :  
`alert("JS...easy !");`

## Afficher un message

- Affiche une fenêtre popup:

```
alert("Bienvenue en JavaScript");
```

- Afficher dans la console

```
console.log("Hello JavaScript");
```

## Indenter

### Indenter

- Une bonne pratique est d'effectuer un retrait vers la droite équivalent à une tabulation à chaque fois qu'on écrit une nouvelle ligne de code à l'intérieur d'une instruction JavaScript.

```
if (age >= 18) {  
    console.log("Accès autorisé");  
  
    if (age >= 21) {  
        console.log("Accès complet");  
    }  
}
```

## Commenter

//: Commentaire sur une ligne

/\* ..... \*/ :Commentaire sur plusieurs lignes

# Variables

- une variable JavaScript est un conteneur servant à stocker temporairement une information.
- Par convention, les noms des variables JavaScript suivent les règles du **camelCase** : tous les mots sont collés avec une majuscule en début de chaque mot, hormis le premier.
- Le nom des variables est **sensible à la casse**, c'est à dire qu'une variable portant le nom a n'est pas la même que celle qui porte le nom A.
- monNomDeVariable respecte le camelCase.
- Pour observer nos variables nous les afficherons dans la console de votre navigateur.
- Un bloc en JavaScript est délimité par des accolades { }.

## Déclaration des variables

- Il faudra commencer par **déclarer** (=créer) les variables avant de pouvoir y affecter une valeur grâce à l'opérateur d'affectation =.
- Il y a trois manières de déclarer une variable en JavaScript : **let**, **const** et **var**.

## L'instruction let

- L'instruction let permet de déclarer une variable de manière globale ou locale de manière simple.

```
<script> let maVar = false; console.log(maVar); </script>
```

- Il est possible de déclarer une variable puis plus tard de réaliser une affectation.

```
<script>
  let maVar2;
  console.log(maVar2); /* undefined */
  maVar2 = 3; console.log(maVar2);
</script>
```

## L'instruction let

- Comme c'est une variable, on peut modifier sa valeur après sa déclaration :

```
<script> let maVar=true; console.log(maVar); maVar=maVar+maVar;
console.log(maVar); </script> /*true + true → 1 + 1 → 2true + true → 1 + 1 → 2/*
```

## L'instruction let (Déclaration variable globale)

- **globale** si on déclare la variable au début d'élément script

```
<script>  
let maVar=1; // globale  
console.log(maVar);  
if (true) { maVar+=2; //locale  
console.log(maVar); }  
maVar+=1; console.log(maVar);  
</script>
```

## L'instruction let (Déclaration variable locale)

- **locale** si on déclare la variable à l'intérieur bloc courant. C'est à dire **entre deux accolades à l'intérieur d'une fonction**, d'un if, d'un for

```
<script> if (true) { let maVara=2;  
console.log(maVara); } maVar+=1;  
console.log(maVara); </script>
```



## L'instruction const

- L'instruction const permet de déclarer une variable globale ou locale constante et donc elle ne pourra pas être modifiée.
- `<script> const maVar4; </script>`
- À l'inverse de l'instruction let, l'instruction const a besoin d'être déclarée avec une valeur initiale.
- Écrire le code suivant :
- `<script> const maVar4; </script>` (problème initialisation)

## L'instruction var

- L'instruction var est la variable qui au début du JavaScript permettait de déclarer des variables.
- Elle est à éviter depuis 2015.
- La variable var est **globale**
- Utilisez à la place les instructions let et const.
- **L'instruction var permet de :**
  - Déclarer une variable et lui assigner une valeur immédiatement :  
`var x = 12;`
  - Déclarer une variable sans lui assigner de valeur et lui assigner une valeur plus tard :  
`var temps; temps = 2;`
  - Déclarer plusieurs variables en même temps :  
`var x = 12, prénom = "jean", nom = "neymar";`

## Types de données

- **Number** : nombres entiers ou flottants → let age = 30;
- **String** : chaînes de caractères → let nom = « Mohamed »;
- **Boolean** : vrai/faux → let actif = true;
- **Null** : valeur vide intentionnelle → let vide = null;
- **Undefined** : variable déclarée sans valeur → let test;

## Opérateurs arithmétiques

- + addition
- - soustraction
- \* multiplication
- / division
- % modulo
- \*\* puissance

## Opérateurs logiques

- && ET logique
- || OU logique
- ! NON logique

```
true && false; // false  
true || false; // true  
!true;        // false
```

## Comparaison (== vs ===)

- == : compare les valeurs (conversion implicite).
- === : compare valeur et type (strict).

```
5 == "5"; // true  
5 === "5"; // false
```

## Les structures conditionnelles

- Elles permettent au programme de **prendre des décisions**.
-  *Si une condition est vraie → ALORS exécuter un code*
- Syntaxe générale:

```
if (condition) {  
    // code exécuté si condition vraie  
} else {  
    // code exécuté sinon  
}
```

### Exemple:

```
let age = 18;  
if (age >= 18) { console.log("Majeur"); } else  
{ console.log("Mineur"); }
```

## Conditions (if, else, else if)

- `if (condition) { ... } else if (autreCondition) { ... } else { ... }`

```
let note = 12;  
  
if (note >= 16) {  
    console.log("Très bien");  
} else if (note >= 10) {  
    console.log("Admis");  
} else {  
    console.log("Ajourné");  
}
```

## Conditions combinées

```
let age = 25;
let permis = true;

if (age >= 18 && permis === true) {
  console.log("Peut conduire");
}
```

## switch

- **Quand utiliser switch ?**

Quand on teste **une seule variable** avec **plusieurs valeurs possibles**.

Syntaxes:

- **switch** (expression) { case valeur1: instructions; **break**; default: instructions; }

```
let jour = 3;
switch (jour) {
  case 1:
    console.log("Lundi");
    break;
  case 2:
    console.log("Mardi");
    break;
  case 3:
    console.log("Mercredi");
    break;
  default:
    console.log("Jour inconnu");
}
```

**Break:** est obligatoire pour éviter l'exécution des autres cas.

## Les Boucles

### Boucle for:

- `for (let i = 0; i < 5; i++) { console.log(i); }`

### Boucle while:

- `let i = 0; while (i < 5) { console.log(i); i++; }`

(Le code peut s'exécuter 0 une fois.)

### Boucle do...while:

- `let i = 0; do { console.log(i); i++; } while (i < 5);`

(Le code s'exécute au moins une fois.)

## break et continue

- **break** : sort de la boucle.
- **continue** : passe à l'itération suivante.

# Les fonctions

- **Problème sans fonction**
- `console.log("Bonjour Ali");`  
`console.log("Bonjour Sami");`  
`console.log("Bonjour Sara");`
- ✗ Répétition du code
- ✗ Difficile à maintenir

**Solution avec fonction**  
`function direBonjour(nom) {`  
 `console.log("Bonjour " +`  
 `nom); }`

**Appel de la fonction:**  
`direBonjour("Ali");`  
`direBonjour("Sami");`

## Déclaration d'une fonction

- Syntaxe générale:

```
function nomFonction() {  
  // instructions  
}
```

- Exemple fonction

## Exemple du fonction

Fonctions sans paramètres

```
function afficherMessage() {  
  console.log("Bienvenue en JavaScript");  
}  
  
afficherMessage();
```

Fonctions avec paramètres

```
function somme(a, b) {  
  console.log(a + b);  
}  
  
somme(5, 3);
```

## Exemple du fonction

Fonction qui retourne une valeur

```
function carre(x) {  
  return x * x;  
}  
  
let resultat = carre(4);  
console.log(resultat);
```

Fonctions anonymes (Fonction sans nom, stockée dans une variable)

```
let addition = function(a, b) {  
  return a + b;  
};  
  
console.log(addition(3, 4));
```



## Exemple du fonction

- Fonctions fléchées

```
const somme = (a, b) => {  
  return a + b;  
};
```

- Version courte:

```
const somme = (a, b) => a + b;
```

Très utilisée en React

## Les tableaux

### Déclaration des tableaux

Un tableau est une structure qui permet de stocker plusieurs valeurs dans une seule variable.

- `let fruits = ["pomme", "banane", "orange"];`
- Les indices commencent à 0

## Parcours de tableaux

- Avec une boucle for

```
for (let i = 0; i < notes.length; i++) {
  console.log(notes[i]);
}
```

- for...of (

```
for (let note of notes) {
  console.log(note);
}
```

## Méthodes utiles

### +Ajouter des éléments

```
notes.push(20); // fin
notes.unshift(8); // début
```

### — Supprimer des éléments

```
notes.pop(); // fin
notes.shift(); // début
```

### ? Rechercher

```
console.log(notes.includes(15)); // true
```

### map (transformation)

```
let notesPlus1 = notes.map(note => note + 1);
```

## Méthodes utiles

### filter (filtrer)

```
let admis = notes.filter(note => note >= 10);
```

## Objets : propriétés et méthodes

- Un **objet** permet de regrouper des données liées.

```
let etudiant = {  
  nom: "Ali",  
  age: 20,  
  moyenne: 14.5  
};
```

### Accès aux données

```
console.log(etudiant.nom);  
console.log(etudiant["age"]);
```

### Différence tableau vs objet

- **Tableau** : indexé numériquement.
- **Objet** : indexé par des clés.

# Objets : propriétés et méthodes

## Méthodes dans un objet

```
let personne = {  
  nom: "Sami",  
  saluer: function() {  
    console.log("Bonjour " + this.nom);  
  }  
};  
  
personne.saluer();
```

## Tableau d'objets

```
let etudiants = [  
  { nom: "Ali", note: 14 },  
  { nom: "Sara", note: 9 },  
  { nom: "Sami", note: 17 }  
];
```

Parcourir:

```
for (let e of etudiants) {  
  console.log(e.nom + " : " + e.note);  
}
```

## Manipulation du DOM

### Qu'est-ce que le DOM ?

- **DOM** = Document Object Model
- Représentation **arborescente** du document
- HTML Créé par le navigateur quand la page se charge
- JavaScript peut **lire, modifier, ajouter, supprimer** n'importe quel nœud

# Qu'est-ce que le DOM ?

- HTML :

```
<p id="txt">Bonjour</p>
```

- DOM :

```
document
├── html
│   └── body
│       └── p (id="txt")
```

☑ Grâce au DOM, JavaScript peut :

- lire le contenu
- modifier le texte ou le style
- ajouter/supprimer des éléments
- réagir aux événements (clic, clavier...)

## Fonctionnalité du Dom

1- Accéder  
aux éléments  
du DOM

2-Lire le  
contenu

3-Lire et modifier  
le contenu

5-Créer et ajouter  
des éléments

4-Modifier le  
style CSS

# Accéder aux éléments du DOM

## 1-Par ID (le plus utilisé)

```
document.getElementById("txt");
```

## 2-Par classe

```
document.getElementsByClassName("box");
```

## 3- Par balise

```
document.getElementsByTagName("p");
```

## 4- Sélecteurs CSS

```
document.querySelector("p"); // premier
document.querySelectorAll(".box"); // tous
```

# Exemple: Accéder aux éléments du DOM

```
<body>
<!-- h1 -->
<h1>Mon titre principal</h1>
<h1>Deuxième titre</h1>
<!-- paragraphes -->
<p>Paragraphe 1</p>
<p>Paragraphe 2</p>
<!-- bouton avec ID -->
<button id="monBouton">Clique ici</button>
<!-- éléments avec classe -->
<div class="item">Item 1</div>
<div class="item">Item 2</div>
<!-- liens -->
<a href="#">Lien A</a>
<a href="#">Lien B</a>
```

Exemple fichier html

```
// Méthodes les plus courantes
const titre = document.querySelector('h1');
// premier <h1> C'est un HTMLElement.

const paragraphes = document.querySelectorAll('p');
// tous les <p> Une NodeList (liste statique)

const btn = document.getElementById('monBouton');
// par ID, Objet unique (HTMLElement).

const items = document.getElementsByClassName('item');
// par classe (HTMLCollection)

const liens = document.getElementsByTagName('a');
// par balise (HTMLCollection)
```

Fichier .js

## Exemple: Accéder aux éléments du DOM

| Variable    | Contenu précis        | Type           |
|-------------|-----------------------|----------------|
| titre       | 1 objet <h1>          | HTML<Element>  |
| paragraphes | Liste de <p>          | NodeList       |
| btn         | 1 bouton              | HTML<Element>  |
| items       | Liste dynamique .item | HTMLCollection |
| liens       | Liste dynamique <a>   | HTMLCollection |

### Caractéristiques liste

C'est une **liste d'éléments**.  
Elle est généralement **statique** (ne change pas si le DOM change).  
On peut utiliser directement

### Caractéristiques collection

C'est une **collection vivante** (live).  
Elle se met à jour automatiquement si on ajoute/supprime des éléments.  
✗ Pas de forEach() direct.

## Lire le contenu

| Propriété / Méthode              | Ce qu'elle renvoie                  |
|----------------------------------|-------------------------------------|
| <code>element.textContent</code> | Le texte pur (sans balises)         |
| <code>element.innerHTML</code>   | Le HTML interne (balises incluses)  |
| <code>element.innerText</code>   | Texte visible (respecte le CSS)     |
| <code>element.value</code>       | Valeur d'un input, textarea, select |



## Lire le contenu (textContent)

- Lire le contenu texte:

```
console.log(titre.textContent); //Affiche: Mon titre principal
```

- Lire le texte des paragraphes (NodeList)

```
paragraphes.forEach(p => {  
  console.log(p.textContent);  
}); /* Affiche: Paragraphe 1  
      Paragraphe 2 */
```

- Lire le texte du bouton

```
console.log(btn.textContent); // Affiche Cliquez ici
```

## Lire le contenu (textContent)

- Lire le contenu des items (HTMLCollection)

```
for(let i = 0; i < items.length; i++){  
  console.log(items[i].textContent);  
}  
/* Affiche Item 1  
   Item 2 */
```

## Lire le contenu

| Propriété / Méthode              | Ce qu'elle renvoie                  | Exemple                                                                                 |
|----------------------------------|-------------------------------------|-----------------------------------------------------------------------------------------|
| <code>element.innerHTML</code>   | Le HTML interne (balises incluses)  | titre. <code>innerHTML</code> → <code>&lt;h1&gt;Mon titre principale &lt;/h1&gt;</code> |
| <code>element.textContent</code> | Le texte pur (sans balises)         | titre. <code>textContent</code> → Mon titre principal                                   |
| <code>element.innerText</code>   | Texte visible (respecte le CSS)     | titre. <code>innerText</code> → Mon titre principal                                     |
| <code>element.value</code>       | Valeur d'un input, textarea, select | input. <code>value</code> → ce que l'utilisateur a tapé                                 |

## Lire et modifier le contenu

### 1-Texte

```
element.textContent = "Nouveau texte";
element.innerText = "Visible seulement";
```

### 2-HTML

```
element.innerHTML = "<strong>Texte</strong>";
```

# Modifier le style CSS

## 1-Style direct

```
element.style.color = "red";
element.style.fontSize = "20px";
```

## 2-Classes CSS

```
element.classList.add("active"); // ajout de la classe active a l'élément
element.classList.remove("active");// supprime la classe active de l'élément
element.classList.toggle("active"); /*Si l'élément n'a pas la classe "active", elle
est ajoutée. Si l'élément a déjà la classe "active", elle est retirée. Exemple
Boutons on/off */
```

**Remarque:** Toujours ajoutée, même si l'élément a déjà d'autres classes.

**Ne fait pas de doublon :** si l'élément a déjà la classe "active", elle ne sera pas ajoutée une 2<sup>e</sup> fois.

# Créer et ajouter des éléments

## • Étape 1 : Créer un élément

```
document.createElement("nomDeLaBalise")
```

Exemple :

```
const nouveauParagraphe = document.createElement("p");
```

## • Étape 2 : Ajouter du contenu

On peut ajouter du texte avec `textContent` ou du HTML avec `innerHTML` :

```
nouveauParagraphe.textContent = "Je suis un nouveau paragraphe";
```

## Créer et ajouter des éléments

### •Étape 3 : Ajouter l'élément dans le DOM

Pour l'ajouter dans la page, on utilise par exemple :

**appendChild** → ajoute à la fin  
**prepend** → ajoute au début  
**insertBefore** → ajoute avant un élément précis

Exemple:

```
const container = document.querySelector("body");
container.appendChild(nouveauParagraphe);
```

Maintenant le paragraphe est **visible dans la page** à la fin du body.

## Manipuler les attributs HTML

- Lire un attribut : `getAttribute("nom")`
- Modifier / ajouter un attribut : `setAttribute("nom", "valeur")`
- Supprimer un attribut : `removeAttribute("nom")`

Exemple

Fichier html

```

<button id="btn" disabled>Cliquer</button>
```

Lire modifier un attribut

```
const img = document.getElementById("photo");
console.log(img.getAttribute("src"));
img.setAttribute("src", "photo2.png");
```

Supprimer un attribut

```
const btn = document.getElementById("btn");
btn.removeAttribute("disabled");// bouton devient cliquable
```

# Les Événements

- Un **événement** est une action de l'utilisateur ou du navigateur (clic, saisie, chargement).
- JavaScript peut **écouter** et **réagir** à ces événements.
- Grâce au JavaScript il est possible d'associer des fonctions, des méthodes à des événements tels que le passage de la souris au-dessus d'une zone, le changement d'une valeur,

## Événements courants:

- **click** : clic sur un élément.
- **mouseover** : passage de la souris.
- **submit** : envoi d'un formulaire.
- **Saisie** dans un input (input, change)
- **Chargement** de la page (load)

# Les Événements

- Syntaxe générale:

```
element.addEventListener("nomEvenement", fonction, useCapture);
```

# Les Événements

- Les types d'événements les plus courants

| Type                        | Description               | Exemple          |
|-----------------------------|---------------------------|------------------|
| <b>click</b>                | Clic souris               | <button>         |
| <b>dblclick</b>             | Double-clic               | <div>            |
| <b>mouseover / mouseout</b> | Souris sur / hors élément | <div>            |
| <b>keydown / keyup</b>      | Touche du clavier         | <input>          |
| <b>input</b>                | Texte saisi               | <input>          |
| <b>change</b>               | Valeur modifiée           | <select>         |
| <b>submit</b>               | Formulaire soumis         | <form>           |
| load                        | Page / image chargée      | <window> / <img> |
| Scroll                      | Défilement                | <window>         |

## Obtenir l'élément qui a déclenché l'événement

- Obtenir l'élément qui a déclenché l'événement:
- Dans `addEventListener`, la fonction reçoit un **objet event** :

```
btn.addEventListener("click", function(event) {
  console.log(event.target); // élément cliqué
});
```

Obtenir l'élément qui a déclenché l'événement:

`event.preventDefault()` → empêche le comportement par défaut (ex : soumission d'un formulaire).

`event.stopPropagation()` → empêche la propagation de l'événement aux parents (bubble).

`event.target` → élément exact qui a déclenché l'événement.

# Exemples pratiques

## A. Clic sur un bouton pour changer le texte

```
btn.addEventListener("click", () => {  
  btn.textContent = "Merci !";  
});
```

## B. Survol pour changer la couleur

```
const div = document.querySelector(".item");  
div.addEventListener("mouseover", () => {  
  div.style.backgroundColor = "yellow";  
});
```

## C. Formulaire → empêcher l'envoi

```
const form = document.querySelector("form");  
form.addEventListener("submit", (e) => {  
  e.preventDefault();  
  alert("Formulaire bloqué !");  
});
```